Lab 5: Functions

Lab Overview

This lab is focused on one of the fundamental building blocks of C++ programming: functions. Functions allow to break large programs into smaller, reusable pieces, making code easier to read, maintain, and debug.

Learning Objectives

#	Learning Objective
1	Understand and write function prototypes
2	Differentiate between pass by value and pass by reference
3	Use const parameters and default arguments effectively
4	Write and call lambda (anonymous) functions with different capture modes
5	Understand variable scope: Local and Global
6	Understand variable storage class: Auto and Static
7	Implement recursive functions for mathematical problems
8	Apply function overloading using different parameter counts and types

Submission Details

You are required to submit all the tasks at github classroom. Create a separate file for each task and upload all the files. Files name must be like task_1.cpp. Submit by March 6th.

B-Section: Submit your solutions at https://classroom.github.com/a/d7QQh11Q

C-Section: Submit your solutions at https://classroom.github.com/a/YW7tS6sf

1. Function Prototypes

A function prototype declares a function before its actual definition. It tells the compiler about the function's name, return type, and parameters so the function can be called before it is defined in the file.

> **Syntax:** return_type function_name(parameter_types);

This is especially useful in large programs where functions may be defined in a different order than they are called.

Example Code

```
#include <iostream>
using namespace std;

void greet();    // Function prototype

int main() {
    greet();     // Call before definition
    return 0;
}

void greet() {   // Function definition
    cout << "Hello from function prototype!" << endl;
}
```

Expected Output: Hello from function prototype!

Key Points

- The prototype ends with a semicolon and the definition does not.
- Parameter names in the prototype are optional; only types are required.
- Without a prototype, the compiler will produce an error if a function is called before it is defined.

2. Function Arguments

Arguments allow data to be passed into a function. C++ supports several ways to pass arguments, each with different behaviour and use cases.

2.1 Pass by Value

A copy of the variable is passed to the function. Changes made inside the function do NOT affect the original variable.

```
#include <iostream>
using namespace std;
void modifyValue(int x) {
   x += 5;
   cout << "Inside function: " << x << endl;
}

int main() {
   int a = 10;
```

```
    modifyValue(a);
    cout << "Inside main: " << a << endl;
    return 0;
}
```

Expected Output: Inside function: 15 Inside main: 10

Notice that a remains 10 in main because only a copy was passed.

2.2 Pass by Reference

The actual variable is passed using a reference (&). Changes inside the function directly modify the original variable.

```
#include <iostream>
using namespace std;

void modifyValue(int &x) {   // & means reference
    x += 5;
    cout << "Inside function: " << x << endl;
}

int main() {
    int a = 10;
    modifyValue(a);
    cout << "Inside main: " << a << endl;
    return 0;
}
```

Expected Output: Inside function: 15 Inside main: 15

This time a is 15 in main because the function modified the original variable.

2.3 Const Reference Parameter

A const reference allows the function to read the variable without copying it, while preventing accidental modification. This is efficient for large data types like strings or objects.

```
#include <iostream>
using namespace std;
```

```
void display(const int &num) {
    cout << "Value received: " << num << endl;
    // num = 100;  // ERROR: cannot modify a const reference
}

int main() {
    int a = 50;
    display(a);
    return 0;
}
```

Expected Output: Value received: 50

2.4 Default Arguments

You can assign a default value to a parameter. If the caller does not supply an argument for that parameter, the default value is used.

```
#include <iostream>
using namespace std;

void display(int num = 100) {
    cout << "Number: " << num << endl;
}

int main() {
    display();     // Uses default: 100
    display(50);   // Uses provided: 50
    return 0;
}
```

Expected Output: Number: 100 Number: 50

Important: Default arguments must be placed at the rightmost positions in the parameter list.

Practice Tasks

Task 1: Create a file proto_practice.cpp. Write a prototype for a function square(int n) that returns the square of an integer. Define it after main() and call it from main() with the value 7.

Task 2: Write a function swap(int &a, int &b) that swaps two integers using pass by reference. Call it from main() and print values before and after swapping.

3. Function Return Types

A function can return a value to the caller using the return statement. The return type must match what is declared in the function signature. A function declared void does not return a value.

```
#include <iostream>
using namespace std;

int add(int a, int b) {
    return a + b;
}

double average(double a, double b) {
    return (a + b) / 2.0;
}

bool isEven(int n) {
    return (n % 2 == 0);
}

int main() {
    cout << "Sum: "     << add(10, 5)         << endl;
    cout << "Average: " << average(10.0, 5.0) << endl;
    cout << "Is 4 even? " << isEven(4)        << endl;
    return 0;
}
```

Expected Output: Sum: 15 Average: 7.5 Is 4 even? 1

Practice Tasks

> **Task 3:** Write a function multiply that returns the product of two integers. Pass the arguments be const reference. Call the function in main.

> **Task 4:** Write a bool function isPrime(int n) that returns true if n is a prime number and false otherwise. Test it with several values in main().

4. Lambda Functions

A lambda function is an anonymous function defined inline, without giving it a name. Lambdas are useful for short, one-time operations. They can capture variables from the surrounding scope.

General Syntax: [capture](parameters) -> return_type { body }

4.1 Capture by Value [=]

When capturing by value, the lambda gets a copy of the outer variables. Modifying them inside the lambda does not change the originals.

```
#include <iostream>
using namespace std;

int main() {
    int a = 10, b;
    auto display = [=](int x) {
        cout << "Product is: " << a * x << endl;
    };
    cout<<"Enter a number: ";
    cin>>b;
    display(b); // Calling the lambda function
    return 0;
}
```

Expected Output: Product is: 70

4.2 Capture by Reference [&]

When capturing by reference, the lambda can read and modify the outer variables directly.

```
#include <iostream>
using namespace std;
int main() {
    int a = 10;
    auto doubleIt = [&]() {
        a *= 2;
        cout << "Inside lambda: " << a << endl;
    };
    doubleIt();
    cout << "After lambda: " << a << endl;
    return 0;
}
```

Expected Output: Inside lambda: 20 After lambda: 20

4.3 Mixed Capture [&a, b]

You can mix capture modes: capture specific variables by reference and others by value.

```
#include <iostream>
using namespace std;

int main() {
    int a = 10, b = 20;
    auto mixed = [&a, b]() {
        a *= 2;         // modifies original a
        // b *= 2;      // ERROR: b is captured by value (read only)
        cout << "a = " << a << ", b = " << b << endl;
    };
    mixed();
    cout << "a in main: " << a << endl;
    return 0;
}
```

Expected Output: a = 20, b = 20 a in main: 20

Practice Tasks

Task 5: Write a lambda that captures two integers x and y by value and prints their sum and product. Call it from main().

Task 6: Write a lambda that captures an integer counter by reference and increments it every time it is called. Call it three times and print the counter value after each call.

5. Variable Scope: Local and Global

5.1 Local Scope

A local variable is declared inside a function or block and is accessible only within that specific scope. It is created when the function is called and destroyed when the function exits.

```
#include <iostream>
using namespace std;

void demo() {
    int x = 10;  // Local variable
    cout << "Local variable x = " << x << endl;
}

int main() {
    demo();
    // cout << x;  // Error: x is not accessible here
    return 0;
}
```

Expected Output: Local variable x = 10

Key Points

- Declared inside functions or blocks
- Accessible only within that scope
- Destroyed after function execution

5.2 Global Scope

A global variable is declared outside all functions and is accessible throughout the program.

```
#include <iostream>
using namespace std;

int x = 100;  // Global variable

void display() {
    cout << "Global variable x = " << x << endl;
}

int main() {
    display();
    cout << "Accessing x in main = " << x << endl;
    return 0;
}
```

Expected Output:
Global variable x = 100
Accessing x in main = 100

Key Points

- Declared outside all functions
- Accessible throughout the program
- Exists for the entire program execution

6. Storage Class: Auto and Static

6.1 Auto Variable

The auto keyword defines a local variable with automatic storage duration. It is the default for local variables.

```
#include <iostream>
using namespace std;

void example() {
    int num = 5;  // by default, local variables are in auto storage class.
    num++;
    cout << "Value of num = " << num << endl;
```

```
}
int main() {
    example();
    example();
    return 0;
}
```

Expected Output:
Value of num = 6
Value of num = 6

Key Points

- Default storage class for local variables
- Created when function is called
- Destroyed when function exits
- Does NOT retain value between calls

6.2 Static Variable

A static local variable is initialized only once and retains its value between function calls. Unlike a regular local variable, it is not destroyed when the function exits.

```
#include <iostream>
using namespace std;

void counter() {
    static int count = 0;  // Initialized only once
    count++;
    cout << "Function called " << count << " times" << endl;
}

int main() {
    counter();
    counter();
    counter();
    return 0;
}
```

Expected Output: Function called 1 times Function called 2 times Function called 3 times

Key Points

- static int count = 0 runs only on the first call; subsequent calls skip the initialization.
- The variable count persists in memory for the lifetime of the program.
- It is local in scope (only accessible inside counter()) but global in lifetime.

Practice Tasks

Task 7: Write a function generateID() that uses a static variable to return an incrementing ID starting from 1001. Call it five times and print each ID.

Task 8: Write a function accumulate(int value) that adds each passed value to a static running total and returns the current total. Call it with values 10, 25, and 5, printing the result after each call.

7. Recursion

Recursion is when a function calls itself. Every recursive function must have a base case (a condition that stops the recursion) to prevent infinite loops.

Rule: Always define a base case first. Without it, your program will crash with a stack overflow.

7.1 Factorial

The factorial of n (written n!) is defined as n * (n-1) * (n-2) * ... * 1. By definition, 0! = 1.

```
#include <iostream>
using namespace std;
int factorial(int n) {
    if (n <= 1) return 1;      // Base case
    return n * factorial(n-1); // Recursive call
}
int main() {
    cout << "5! = " << factorial(5) << endl;
    return 0;
}
```

Expected Output: 5! = 120

7.2 Sum from 1 to N

Sum all integers from 1 up to n recursively.

```
#include <iostream>
using namespace std;
int sum(int n) {
    if (n == 0) return 0;   // Base case
    return n + sum(n - 1);  // Recursive call
}
int main() {
    cout << "Sum 1 to 5 = " << sum(5) << endl;
    return 0;
}
```

Expected Output: Sum 1 to 5 = 15

Practice Tasks

Task 9: Write a recursive function fibonacci(int n) that returns the nth Fibonacci number (where fib(0)=0, fib(1)=1, fib(n)=fib(n-1)+fib(n-2)). Print the first 10 Fibonacci numbers in main().

Task 10: Write a recursive function power(int base, int exp) that computes base raised to the power of exp. Test with power(2, 8) and power(3, 4).

8. Function Overloading

Function overloading allows you to define multiple functions with the same name, as long as they differ in the number or type of their parameters. The compiler automatically selects the correct version based on the arguments you pass. This is known as compile-time polymorphism (static polymorphism).

8.1 Overloading by Number of Arguments

```
#include <iostream>
using namespace std;
int add(int a, int b)        { return a + b; }
int add(int a, int b, int c)  { return a + b + c; }
int main() {
    cout << "Two args:   " << add(10, 5)       << endl;
    cout << "Three args: " << add(10, 10, 10)  << endl;
    return 0;
}
```

Expected Output: Two args: 15 Three args: 30

8.2 Overloading by Type of Arguments

```
#include <iostream>
using namespace std;
void display(int a)    { cout << "Integer: " << a   << endl; }
void display(double a) { cout << "Double:  " << a   << endl; }
void display(string s) { cout << "String:  " << s   << endl; }
int main() {
    display(10);
    display(10.5);
    display("Hello");
    return 0;
}
```

Expected Output: Integer: 10 Double: 10.5 String: Hello

Key Points

- Functions cannot be overloaded based only on return type — the parameter list must differ.
- The compiler determines which overload to call at compile time.
- Overloading improves readability by allowing the same logical name for related operations.

Practice Tasks

Task 11: Create an overloaded function area() with three versions: area(int side) for a square, area(int length, int width) for a rectangle, and area(double radius) for a circle (use 3.14159 * r * r). Call all three from main().

Task 12: Create an overloaded function printType() that accepts int, double, char, and string and prints a message indicating the data type of the argument received. Test with one call of each type.

9. Challenge Exercise

Mini Calculator Program

Build a program that implements the following requirements:

- Use function prototypes for all functions declared before main().
- Write overloaded calculate(int, int, char) and calculate(double, double, char) functions that perform +, -, *, / based on the operator character.
- Write a recursive function power(double base, int exp) for exponentiation.
- Write a lambda inside main() that takes two numbers and prints their absolute difference.
- Use a static variable inside a function total_calc() to count how many calculations have been performed, and print the count at the end.

Hint: For division, check if the divisor is zero before dividing and print an error message if so.

Expected sample output (will vary based on your inputs):

```
10 + 5 = 15
10.5 - 3.2 = 7.3
2 ^ 8 = 256
Absolute difference: 5
Total calculations performed: 3
```

10. Concept Summary

Concept	Key Syntax / Feature	When to Use
Function Prototype	return_type name(types);	Declare before definition
Pass by Value	void f(int x)	When original must not change
Pass by Reference	void f(int &x)	When function should modify original
Const Reference	void f(const int &x)	Read-only access, no copying
Default Arguments	void f(int x = 10)	Optional parameters with defaults
Lambda (by value)	auto f = [=](){ };	Capture snapshot of variables
Lambda (by ref)	auto f = [&](){ };	Capture and modify outer variables
Static Variable	static int count = 0;	Persist value across function calls
Recursion	return f(n-1);	Problems with self-similar structure
Function Overloading	Same name, different params	Same operation on different types

Common Mistakes to Avoid

- Forgetting the semicolon at the end of a function prototype.
- Using pass by value when you intend to modify the original variable.
- Writing a recursive function without a base case (causes infinite recursion and a crash).
- Placing non-default parameters after default parameters in a function signature.
- Trying to overload functions that differ only in return type.
- Modifying a variable captured by value in a lambda (compile error without mutable keyword).